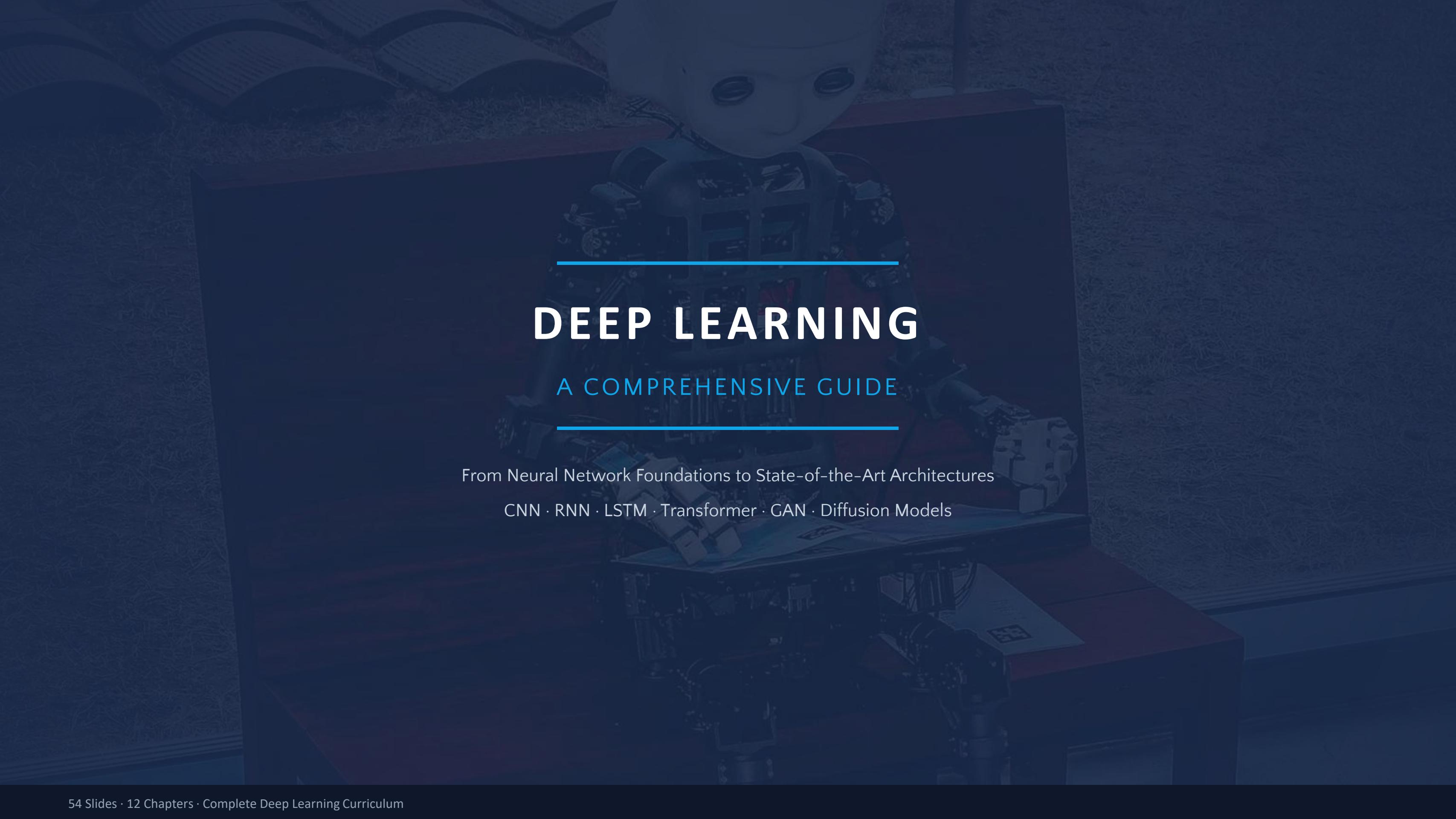




DEEP LEARNING

A COMPREHENSIVE GUIDE

From Neural Network Foundations to State-of-the-Art Architectures

CNN · RNN · LSTM · Transformer · GAN · Diffusion Models

Presentation Overview

01 Introduction to Deep Learning Foundations & Definitions	02 Neural Network Fundamentals Structure & Learning Process	03 Feedforward Neural Networks FNN Architecture	04 Convolutional Neural Networks Computer Vision
05 Recurrent Neural Networks RNN, Sequential Data	06 Long Short-Term Memory LSTM & GRU	07 Autoencoders AE & VAE	08 Transformer Models Attention Mechanisms
09 Generative Models GANs & Diffusion	10 Encoder-Decoder Seq2Seq Architecture	11 The DL Pipeline Data to Deployment	12 Applications & Impact Real-World Use Cases

This presentation covers **12 major chapters** spanning from foundational concepts to cutting-edge architectures, providing a complete deep learning curriculum with practical applications.

06

Long Short-Term Memory

Gated Architecture for Long-Range Dependencies

LSTM Architecture Overview

LSTM (Long Short-Term Memory) is an advanced RNN variant capable of learning long-term dependencies. Introduced by Hochreiter & Schmidhuber (1997). Key innovation: [gating mechanisms](#) that regulate information flow.

Input Gate

Controls what **new information** enters the cell state. Decides which values to update and what candidate values to consider.

Forget Gate

Decides what information to **discard** from the previous cell state. Outputs 0 (forget) to 1 (keep) for each element.

Output Gate

Determines what parts of the cell state to **output** as the next hidden state. Filters relevant information for the current prediction.

Memory Cell

The **cell state** (C_t) acts as a "conveyor belt" — information flows through largely unchanged, with only minor linear interactions.

The Conveyor Belt Analogy

Imagine a **conveyor belt** (cell state) running through a factory with several gates:

Forget Gate = Quality inspector removes defective items from the belt

Input Gate = Worker adds new processed items onto the belt

Output Gate = Dispatcher selects relevant items to send to the next station

The belt itself runs straight with minimal changes — this linear path is why LSTMs avoid the vanishing gradient problem. The gates control what gets on, off, and through.

LSTM Gates: Forget & Input

Forget Gate

Computes a 0-to-1 value using **sigmoid** over h_{t-1} and x_t .

Near 0 = discard information

Near 1 = keep information

Formula:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

This gate decides what past information is no longer relevant — like clearing old context when a new topic begins.

Input Gate

Two-step process for updating the cell state:

Step 1: Sigmoid decides which values to update

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Step 2: tanh creates candidate values

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Together they determine what new information to store in the cell state.

Cell State Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

How it works: The old cell state C_{t-1} is multiplied element-wise by the forget gate f_t (discarding selected information). Then the input gate i_t is multiplied element-wise by the candidate values $\tilde{C}_t$ (adding new information). The result is the updated cell state C_t .

This **additive update** (rather than multiplicative like in standard RNNs) is why LSTMs avoid the vanishing gradient problem — gradients can flow through the cell state via the unmodified path.

LSTM Output Gate & Cell State

Output Gate

Determines the next hidden state (which also = output).

Step 1: Sigmoid decides which parts to output

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Step 2: Cell state through tanh

$$\text{Step 3: } h_t = o_t \odot \tanh(C_t)$$

The Logic of Information Flow

The cell state C_t maintains a vector of the same dimensionality as the hidden state.

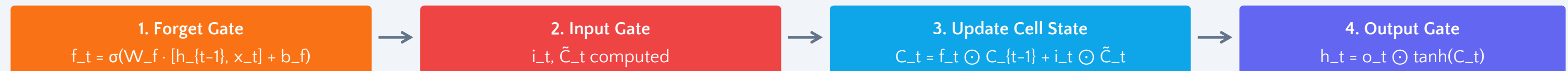
Forget Gate removes existing information

Input Gate adds new relevant information

Output Gate filters what becomes the hidden state

Together they enable selective memory — keeping, discarding, and updating information precisely.

Complete LSTM Forward Pass Summary



Key Insight: The cell state C_t runs through the entire chain with minimal interactions — only element-wise multiplication and addition. This linear path allows gradients to flow unchanged, solving the vanishing gradient problem.

The gates (sigmoid outputs between 0 and 1) act as **differentiable switches** — they can be learned through backpropagation just like any other neural network parameter. During training, the LSTM learns which information to remember, forget, and output for each task.

LSTM Applications & Bi-LSTM



Speech Recognition

Language and acoustic modeling for voice assistants and transcription.



Sequence Labeling

POS tagging, Named Entity Recognition (NER), phrase chunking.



Image Captioning

CNN encodes image, LSTM generates natural language description.



Machine Translation

Sequence-to-sequence models translating between languages.



Video Captioning

Input sequence of CNN frames generates video descriptions.



Text Classification

Sentiment analysis, topic categorization, spam detection.

Bi-directional LSTM (Bi-LSTM)

Standard LSTMs process sequences in one direction (left-to-right). Bi-LSTM adds a **second LSTM** that processes the sequence in reverse (right-to-left).

How it works:

- Forward LSTM processes $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$, producing hidden states $h_{\rightarrow_1} \dots h_{\rightarrow_T}$
- Backward LSTM processes $x_T \rightarrow x_{\{T-1\}} \rightarrow \dots \rightarrow x_1$, producing hidden states $h_{\leftarrow_T} \dots h_{\leftarrow_1}$
- At each position, outputs are **concatenated**: $h_t = [h_{\rightarrow_t}, h_{\leftarrow_t}]$

Advantage: Captures context from **both past and future** simultaneously. Critical for tasks like NER where a word's meaning depends on words both before and after it.

Gated Recurrent Units (GRU)

GRU is a simplified variant of LSTM (Cho et al., 2014). Combines forget and input gates into a single **update gate** and merges cell state with hidden state.

Update Gate (z)

Controls how much **past information** to keep.

Combines LSTM's forget + input gates into one.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

z near 1: keep most past info. z near 0: use mostly new candidate.

Reset Gate (r)

Controls how much **past information** to forget when computing new candidate.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

r near 0: ignore past, compute fresh candidate from input only.

r near 1: include full past context.

LSTM vs GRU Comparison

Feature	LSTM	GRU
Gates	3 gates (input, forget, output)	2 gates (update, reset)
Cell State	Separate cell + hidden state	Single hidden state only
Parameters	More parameters (4x per gate)	Fewer parameters (3x per gate)
Training Speed	Slower (more computation)	Faster (fewer params)
Performance	Better on very long sequences	Comparable on most tasks

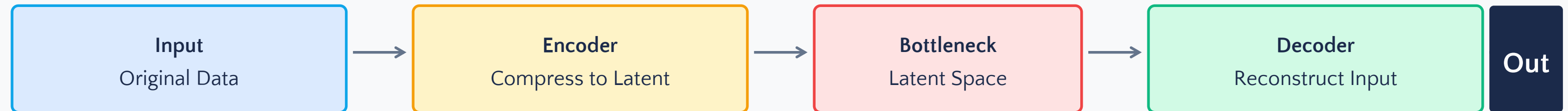
07

Autoencoders

Unsupervised Learning, Compression & Generative Modeling

Autoencoder Architecture

Autoencoders **compress (encode)** input data then **reconstruct (decode)** the original input from the compressed representation.



Training Objective

Minimize **reconstruction loss** — the divergence between reconstructed output and original input.

Forces the model to learn weights that retain only the **most essential features**.

The latent space (bottleneck) captures the most meaningful subset of input features.

Use Cases

- **Data Compression** — efficient representation
- **Dimensionality Reduction** — fewer features
- **Feature Extraction** — meaningful representations
- **Denoising** — reconstruct clean from corrupted
- **Fraud Detection** — high error = anomaly

Key Insight

The bottleneck forces an **information bottleneck** — the network must learn to compress the input into a lower-dimensional representation that still contains enough information for reconstruction.

After training, the **decoder is often discarded** — the encoder is the valuable component. It transforms raw data into a compact, meaningful latent representation useful for downstream tasks.

The dimensionality of the latent space is a critical hyperparameter: too small = information loss; too large = no compression benefit.

Variational Autoencoders (VAE)

VAEs are a type of generative model that extends autoencoders. Instead of learning a fixed latent representation, VAEs learn a [probability distribution](#) over the latent space.

VAE Architecture

[Encoder](#) outputs two vectors:

- **Mean (μ)** — center of the Gaussian distribution
- **Standard Deviation (σ)** — spread of the distribution

[Sampling](#) — draw a latent vector z from $N(\mu, \sigma^2)$

[Decoder](#) reconstructs input from sampled z

Reparameterization Trick: $z = \mu + \sigma \cdot \epsilon$ where $\epsilon \sim N(0,1)$

Key Properties

Generative Capability: Can generate new data by sampling from the learned latent distribution and passing through the decoder.

Smooth Latent Space: Nearby points in latent space produce similar outputs — enables interpolation.

Training Stability: More stable than GANs due to the probabilistic framework.

KL Divergence Loss: Regularizes the learned distribution to be close to standard normal $N(0,1)$.

VAE Loss Function

$$\text{Loss} = \text{Reconstruction Loss} + \text{KL Divergence}(q(z|x) \parallel p(z))$$

Applications

[Image Generation](#) — generate new images by sampling from latent space. [Data Augmentation](#) — create synthetic training samples. [Anomaly Detection](#) — high reconstruction error indicates outliers. [Disentangled Representations](#) — separate factors of variation (e.g., pose vs identity in faces). [Drug Discovery](#) — generating molecular structures with desired properties.

10

Encoder-Decoder Architecture

Sequence-to-Sequence Learning & Neural Machine Translation

Sequence-to-Sequence Learning

Encoder-Decoder architecture handles input and output sequences of **different lengths** — the foundation of neural machine translation.

Encoder

Processes the **input sequence** (e.g., English sentence) and compresses it into a fixed-size **context vector** ("thought vector").

Typically uses RNN, LSTM, or GRU layers.

The final hidden state of the encoder contains a compressed representation of the entire input sequence.

Decoder

Takes the context vector and **generates output sequence** (e.g., Arabic translation) one token at a time.

Also uses RNN/LSTM/GRU with attention to encoder output.

At each step, the internal state updates based on current input and previous state.

Pipeline: **Input Sequence** → Encoder → **Context Vector** → Decoder → **Output Sequence** (token by token)

Applications

Machine Translation — English to Arabic, Chinese to French

Text Summarization — long articles to brief summaries

Chatbots & Question Answering — conversational AI

Image Captioning — CNN encoder + LSTM decoder

Speech Recognition — audio sequence to text transcript

Training & Inference in Seq2Seq

Training Phase

Model learns to map input sequences to target sequences.

Teacher Forcing:

During training, the model receives **ground truth target tokens** as input at each step, regardless of what it predicted previously.

This accelerates convergence by providing correct context.

Parameters adjusted via gradient descent to minimize loss (typically cross-entropy).

Training continues iteratively until satisfactory performance.

Inference Phase

Trained model generates outputs for **unseen inputs**.

No Ground Truth:

No access to target tokens during generation.

Generates output **one token at a time** based on previously generated tokens.

Beam Search:

Explores multiple candidate sequences simultaneously, keeping top-k most promising paths at each step for higher quality outputs.

Key Challenge: Information Bottleneck

The encoder must compress the entire input sequence into a **single fixed-size context vector**. This creates an information bottleneck — especially problematic for long sequences where critical details may be lost.

Solution: Attention Mechanism

Instead of relying solely on the final encoder state, the decoder can **attend to all encoder hidden states** at each decoding step. This allows the model to focus on relevant parts of the input dynamically, dramatically improving translation quality. The Attention mechanism (Bahdanau et al., 2015) was the key innovation that made seq2seq models practical.